CAPSTONE PROJECT:

AgriConnect Farm-to-Market Platform with Mobile Money Wallet

One project, built phase-by-phase across the whole semester. Every 's lecture topic becomes a build step. Teams of 2–4 students. All code is MySQL 8.0+.

1. THE BUSINESS PROBLEM

In rural Uganda, farmers lose 30–40% of produce value because they sell cheaply to middlemen at the farm gate, buyers can't find reliable suppliers, and payment is risky (cash, no records). A farmer in Mbale has matooke but doesn't know Kampala prices; a Kampala restaurant wants consistent supply but can't trust remote sellers.

Your solution: A platform where farmers list produce, buyers order and pay from a mobile-money-style wallet, the platform takes a 5% commission, and deliveries are tracked. You will build the entire database that powers this business.

2. BUSINESS RULES (your requirements document)

1. A user can be a farmer, buyer, or both identified uniquely by phone number.
2. Every user has exactly one wallet; wallet balance can never go below zero.
3. All money movements (cash-in, cash-out, payment, receipt) must be permanently logged.
4. An order may contain several products; prices are frozen at purchase time (even if the farmer later changes price).
5. You cannot sell more stock than exists even if two buyers order the last bag at the same second.
6. Payment is atomic: wallet debit, stock reduction, and farmer credit all succeed together or not at all.
7. Platform keeps 5% commission on every order.
8. Buyers rate products 1–5 only for items they actually purchased.
9. Product details vary by type (grain has variety, livestock has weight) schema must flex without redesign.

10. Admins need reports: monthly sales, top farmers, best products, buyer growth.

3. SEMESTER ROADMAP each = one phase

ACTIVITY	Course topic	Project phase
0–1	Environment setup	Phase 0: Install MySQL, create repo, write requirements
2	ER Modeling	Phase 1: ER diagram
3	Relational model & algebra	Phase 2: Map ER → tables, write algebra
4	SQL fundamentals & joins	Phase 3: DDL + seed data, basic queries
5	Advanced SQL	Phase 4: CTEs, window functions, views
6	Normalization	Phase 5: Normalize audit + denormalize decision
7	Transactions/ACID	Phase 6: Checkout stored procedure
8	Concurrency & locking	Phase 7: Break it, then fix it (locks, deadlock)
9	Indexing	Phase 8: Index + EXPLAIN evidence
10	Query optimization	Phase 9: Rewrite slow queries, prove speedup
11	Physical design	Phase 10: Partition the analytics table
12	Distributed DBs	Phase 11: Replication design doc
13	NoSQL	Phase 12: JSON catalog extension
14	Integration	Phase 13: Final docs, demo, defense

PHASE 0 (0–1): Setup & Requirements

- Install MySQL 8 + MySQL Workbench; create a Git repo `agriconnect-db`.
- Interview one real vendor/farmer (or role-play) and refine the 10 business rules above.
- Create the database shell:

```sql
CREATE DATABASE agriconnect CHARACTER SET utf8mb4;

USE agriconnect;
```

Deliverable: 1-page requirements document.

PHASE 1 (2): ER Diagram

Entities: `USERS`, `TOWNS`, `PRODUCTS`, `WALLETS`, `WALLET_TXNS`, `ORDERS`, `ORDER_ITEMS`, `DELIVERIES`, `REVIEWS`.

Relationships & cardinalities to show on the diagram:

Relationship	**Cardinality**
TOWN – USERS	1 : M
USER (farmer) – PRODUCTS	1 : M
USER – WALLET	01:01
WALLET – WALLET_TXNS	1 : M
USER (buyer) – ORDERS	1 : M
ORDER – ORDER_ITEMS	1 : M
PRODUCT – ORDER_ITEMS	1 : M *(resolves M:N between ORDERS and PRODUCTS)*
ORDER – DELIVERY	01:01
USER (buyer) – REVIEWS – PRODUCT	M : N *(resolved by REVIEWS as associative entity)*

Note `WALLET_TXNS` and `ORDER_ITEMS` are weak entities (existence-dependent). Draw in draw.io or Workbench.

Deliverable: ERD image (Assignment 1 material).

PHASE 2 (3): Mapping + Relational Algebra

Mapping rules applied: strong entities → tables; M:N → junction table (`order_items`); 1:1 → shared PK (`wallets.user_id` UNIQUE); multivalued/weak → dependent tables.

Write 5 report requirements in algebra, then verify each in SQL:

```text
Q1: π product_name (σ category='FRUIT' (Products))

Q2: π full_name (Users ⋈ σ balance > 500000 (Wallets))

Q3: π town_name, COUNT(order_id) (Towns ⋈ Users ⋈ Orders  GROUP town)

Q4: π product_name (σ rating = 5 (Reviews ⋈ Products))

Q5: π product_name, SUM(line_total) (Products ⋈ Order_Items  GROUP product)
```

Deliverable: mapping worksheet + 5 verified SQL equivalents.

PHASE 3 (4): Schema Implementation + Seed Data

```sql
CREATE TABLE towns (
  town_id   INT AUTO_INCREMENT PRIMARY KEY,
  town_name VARCHAR(60) NOT NULL UNIQUE,
  district  VARCHAR(60)
);

CREATE TABLE users (
  user_id    INT AUTO_INCREMENT PRIMARY KEY,
  full_name  VARCHAR(100) NOT NULL,
  phone      VARCHAR(13) NOT NULL UNIQUE,          -- 256XXXXXXXXX
  email      VARCHAR(120) UNIQUE,
  role       ENUM('FARMER','BUYER','BOTH') NOT NULL DEFAULT 'BUYER',
  town_id    INT NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
  FOREIGN KEY (town_id) REFERENCES towns(town_id)
);

CREATE TABLE wallets (
  wallet_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id   INT NOT NULL UNIQUE,                -- 1:1 with users
  balance   DECIMAL(14,2) NOT NULL DEFAULT 0 CHECK (balance >= 0),
  FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE wallet_txns (
  txn_id     BIGINT AUTO_INCREMENT PRIMARY KEY,
  wallet_id  INT NOT NULL,
  txn_type   ENUM('CASH_IN','CASH_OUT','PAYMENT','RECEIPT') NOT NULL,
  amount     DECIMAL(12,2) NOT NULL CHECK (amount > 0),
  ref_code   VARCHAR(20) UNIQUE,
  counterparty_wallet INT NULL,
  txn_date   DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (wallet_id) REFERENCES wallets(wallet_id)
);

CREATE TABLE products (
  product_id   INT AUTO_INCREMENT PRIMARY KEY,
  farmer_id    INT NOT NULL,
```

```
  product_name VARCHAR(100) NOT NULL,
  category      ENUM('GRAIN','FRUIT','VEGETABLE','TUBER','LIVESTOCK','OTHER')
NOT NULL,
  unit      ENUM('kg','crate','bag','bunch','litre') NOT NULL,
  unit_price   DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
  stock_qty    DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
  is_active    BOOLEAN DEFAULT TRUE,
  FOREIGN KEY (farmer_id) REFERENCES users(user_id)
);

CREATE TABLE orders (
  order_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
  buyer_id      INT NOT NULL,
  order_date     DATETIME DEFAULT CURRENT_TIMESTAMP,
  status        ENUM('PENDING','PAID','DISPATCHED','DELIVERED','CANCELLED')
                DEFAULT 'PENDING',
  delivery_address VARCHAR(200),
  total_amount    DECIMAL(14,2) NOT NULL DEFAULT 0,
  FOREIGN KEY (buyer_id) REFERENCES users(user_id)
);

CREATE TABLE order_items (
  order_item_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_id     BIGINT NOT NULL,
  product_id    INT NOT NULL,
```

```
  qty          DECIMAL(10,2) NOT NULL CHECK (qty > 0),
  unit_price   DECIMAL(10,2) NOT NULL,          -- frozen at purchase time
  line_total   DECIMAL(12,2) GENERATED ALWAYS AS (qty * unit_price) STORED,
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id)
);

CREATE TABLE deliveries (
  delivery_id  BIGINT AUTO_INCREMENT PRIMARY KEY,
  order_id     BIGINT NOT NULL UNIQUE,          -- 1:1 with orders
  driver_name  VARCHAR(100),
  status       ENUM('SCHEDULED','IN_TRANSIT','DONE','FAILED') DEFAULT 'SCHEDULED',
  dispatched_at DATETIME NULL,
  delivered_at  DATETIME NULL,
  FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

CREATE TABLE reviews (
  review_id  INT AUTO_INCREMENT PRIMARY KEY,
  buyer_id   INT NOT NULL,
  product_id INT NOT NULL,
  rating     TINYINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment    VARCHAR(300),
  review_date DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
  UNIQUE (buyer_id, product_id),
  FOREIGN KEY (buyer_id) REFERENCES users(user_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Seed realistic data (extend to ~50 users, ~100 products):

```sql
INSERT INTO towns (town_name, district) VALUES
('Kampala Central','Kampala'),('Mbale','Mbale'),('Mbarara','Mbarara'),
('Gulu','Gulu'),('Jinja','Jinja');

INSERT INTO users (full_name, phone, role, town_id) VALUES
('Nakato Sarah','256772100001','BOTH',1),
('Okello John','256772100002','FARMER',2),
('Mwesigwa David','256772100003','FARMER',3),
('Achieng Grace','256772100004','BUYER',1),
('Kato Peter','256772100005','BUYER',5);

INSERT INTO wallets (user_id, balance)
SELECT user_id, 500000 FROM users;    -- everyone starts with 500,000 UGX

INSERT INTO products (farmer_id, product_name, category, unit, unit_price, stock_qty) VALUES
(2,'Matooke Bunch','TUBER','bunch',25000,120),
(2,'Groundnuts','GRAIN','kg',7000,300),
```

(3,'Matoke Crates','FRUIT','crate',45000,40),

(3,'Fresh Milk','OTHER','litre',1500,500),

(1,'Ndizi Nshaki','FRUIT','bunch',30000,60);

Deliverable: working `schema.sql` + `seed.sql` scripts.

PHASE 4 (5): The Query Pack business reports

```sql

-- 1. Every farmer and their listed products (INNER JOIN)

SELECT u.full_name, p.product_name, p.unit_price

FROM users u JOIN products p ON u.user_id = p.farmer_id;

-- 2. Buyers with no orders yet (LEFT JOIN + IS NULL)

SELECT u.full_name

FROM users u

LEFT JOIN orders o ON o.buyer_id = u.user_id

WHERE u.role <> 'FARMER' AND o.order_id IS NULL;

-- 3. Total spent per buyer

SELECT u.full_name, SUM(o.total_amount) AS lifetime_spend

FROM users u JOIN orders o ON o.buyer_id = u.user_id

WHERE o.status = 'DELIVERED'

GROUP BY u.user_id, u.full_name;
```

-- 4. Products never ordered (subquery / NOT IN)

SELECT product_name FROM products

WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items);

-- 5. NULL practice: deliveries not yet dispatched

SELECT o.order_id, d.driver_name FROM orders o

LEFT JOIN deliveries d ON d.order_id = o.order_id

WHERE d.dispatched_at IS NULL;

Deliverable: `queries_basic.sql` with 15+ queries (Assignment 2 material).

PHASE 5 (5–6): Advanced SQL CTEs, Windows, Views

```sql

-- Top 3 products per category (window ranking)

WITH prod_sales AS (

  SELECT p.category, p.product_name, SUM(oi.qty) AS total_qty

  FROM order_items oi JOIN products p ON p.product_id = oi.product_id

  GROUP BY p.category, p.product_name

),

ranked AS (

  SELECT , ROW_NUMBER() OVER (PARTITION BY category ORDER BY total_qty DESC) rnk

  FROM prod_sales

)

SELECT category, product_name, total_qty FROM ranked WHERE rnk <= 3;
```

```
-- Month-over-month platform revenue using LAG
WITH monthly AS (
  SELECT DATE_FORMAT(order_date,'%Y-%m') yr_mo, SUM(total_amount) rev
  FROM orders WHERE status = 'DELIVERED'
  GROUP BY yr_mo
)
SELECT yr_mo, rev,
       rev - LAG(rev) OVER (ORDER BY yr_mo) AS change_prev,
       ROUND(100(rev - LAG(rev) OVER (ORDER BY yr_mo))
            / NULLIF(LAG(rev) OVER (ORDER BY yr_mo),0),1) AS pct_change
FROM monthly;

-- Farmer earnings leaderboard with rank
SELECT u.full_name,
       SUM(oi.line_total)  0.95 AS net_earnings,     -- after 5% commission
       RANK() OVER (ORDER BY SUM(oi.line_total) DESC) AS farmer_rank
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
JOIN users u    ON u.user_id = p.farmer_id
GROUP BY u.user_id, u.full_name;

-- Reusable views for the admin dashboard
CREATE VIEW v_farmer_earnings AS
```

```
SELECT p.farmer_id, u.full_name,
       SUM(oi.line_total) AS gross,
       SUM(oi.line_total)  0.95 AS net
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
JOIN users u ON u.user_id = p.farmer_id
GROUP BY p.farmer_id, u.full_name;

CREATE VIEW v_catalog AS
SELECT product_id, product_name, category, unit, unit_price,
       stock_qty > 0 AS in_stock
FROM products WHERE is_active = TRUE;
```

PHASE 6 (6): Normalization Audit

Find the anomalies. Suppose your teammate proposed this single "orders" table:

```sql
-- BAD: everything in one table
CREATE TABLE bad_orders (
  order_id INT, buyer_name VARCHAR(100), buyer_phone VARCHAR(13),
  farmer_name VARCHAR(100), product_name VARCHAR(100), category VARCHAR(20),
  unit_price DECIMAL(10,2), qty DECIMAL(10,2), town VARCHAR(60)
);
```

Document in your report:

- Update anomaly: farmer changes phone → update hundreds of rows.
- Insert anomaly: can't record a new product until someone orders it.
- Delete anomaly: delete the last order → farmer info vanishes.

Show the fix: your Phase 3 schema already achieves 3NF demonstrate the FDs (`order_id → buyer_id`, `product_id → farmer_id, category, unit_price`…) and prove no partial or transitive dependencies.

Justified denormalization: for the dashboard you will later create `fact_order_lines` (Phase 10) duplicating category and names for read speed. Write the trade-off paragraph.

Deliverable: `normalization_audit.pdf` (Assignment 3 material).

PHASE 7 (7): Transactions the Checkout Procedure

This is the heart of the project: one atomic operation touching 5 tables.

```sql
DELIMITER //
CREATE PROCEDURE sp_place_order(
  IN p_buyer_id INT,
  IN p_product_id INT,
  IN p_qty DECIMAL(10,2),
  IN p_address VARCHAR(200)
)
BEGIN
```

```
DECLARE v_price, v_stock, v_total, v_commission DECIMAL(12,2);
DECLARE v_farmer, v_buyer_wallet, v_farmer_wallet INT;
DECLARE v_order_id BIGINT;

DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN ROLLBACK; RESIGNAL; END;

START TRANSACTION;

-- Lock product row: guarantees stock check + deduction are safe
SELECT unit_price, stock_qty, farmer_id INTO v_price, v_stock, v_farmer
FROM products WHERE product_id = p_product_id FOR UPDATE;

IF v_stock < p_qty THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient stock';
END IF;

SET v_total = v_price  p_qty;
SET v_commission = v_total  0.05;

SELECT wallet_id INTO v_buyer_wallet FROM wallets
 WHERE user_id = p_buyer_id FOR UPDATE;

IF (SELECT balance FROM wallets WHERE wallet_id = v_buyer_wallet) < v_total THEN
```

```
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient wallet balance';
END IF;

SELECT wallet_id INTO v_farmer_wallet FROM wallets WHERE user_id = v_farmer;

-- 1. Create the order
INSERT INTO orders (buyer_id, status, delivery_address, total_amount)
VALUES (p_buyer_id, 'PAID', p_address, v_total);
SET v_order_id = LAST_INSERT_ID();

-- 2. Line item with frozen price
INSERT INTO order_items (order_id, product_id, qty, unit_price)
VALUES (v_order_id, p_product_id, p_qty, v_price);

-- 3. Reduce stock
UPDATE products SET stock_qty = stock_qty - p_qty
 WHERE product_id = p_product_id;

-- 4. Move the money
UPDATE wallets SET balance = balance - v_total WHERE wallet_id = v_buyer_wallet;
UPDATE wallets SET balance = balance + (v_total - v_commission)
 WHERE wallet_id = v_farmer_wallet;

-- 5. Immutable money log
```

```
  INSERT INTO wallet_txns (wallet_id, txn_type, amount, ref_code, counterparty_wallet)
  VALUES
(v_buyer_wallet,'PAYMENT',v_total,CONCAT('ORD',v_order_id),v_farmer_wallet),
        (v_farmer_wallet,'RECEIPT',v_total - v_commission,CONCAT('ORD',v_order_id),v_buyer_wallet);

  -- 6. Schedule delivery
  INSERT INTO deliveries (order_id) VALUES (v_order_id);

  COMMIT;
END //
DELIMITER ;

-- Test success:
CALL sp_place_order(4, 1, 3, 'Plot 12, Ntinda, Kampala');

-- Test atomicity: this MUST fail and change NOTHING:
CALL sp_place_order(4, 2, 99999, 'Ntinda');   -- exceeds stock
SELECT  FROM orders ORDER BY order_id DESC LIMIT 1;   -- unchanged
```

Also create the wallet top-up procedure:

```sql
CREATE PROCEDURE sp_cash_in(IN p_user_id INT, IN p_amt DECIMAL(12,2))
BEGIN
```

```
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
  START TRANSACTION;
  UPDATE wallets SET balance = balance + p_amt WHERE user_id = p_user_id;
  INSERT INTO wallet_txns (wallet_id, txn_type, amount, ref_code)
  SELECT wallet_id, 'CASH_IN', p_amt, CONCAT('CI', UNIX_TIMESTAMP())
  FROM wallets WHERE user_id = p_user_id;
  COMMIT;
END //
DELIMITER ;
```

Deliverable: `procedures.sql` + a demo showing rollback on failure (record a video/screenshot).

PHASE 8 (8): Concurrency Break It, Then Fix It

Experiment A oversell without locking. Remove `FOR UPDATE` from a copy of the procedure (`sp_place_order_nolock`). Open two MySQL sessions and run simultaneously for a product with 1 unit left. Both succeed → stock = -1. Screenshot it. That's a lost update.

Experiment B the fix. Restore `FOR UPDATE`: one session waits, then fails with "Insufficient stock". Business rule 5 now holds under any concurrency.

Experiment C deadlock. In two sessions run opposite transfers:

```sql
-- Session 1                    -- Session 2
```

START TRANSACTION; START TRANSACTION;

UPDATE wallets SET balance=balance-1000 WHERE wallet_id=1;

UPDATE wallets SET balance=balance-1000 WHERE wallet_id=2;

UPDATE wallets SET balance=balance+1000 WHERE wallet_id=2; -- waits...

UPDATE wallets SET balance=balance+1000 WHERE wallet_id=1;

-- ERROR 1213: Deadlock found; MySQL kills one

COMMIT; ROLLBACK;

```

Explain in your report: InnoDB detects the wait cycle, rolls back the victim, the app retries.

Isolation levels: run dirty-read/phantom experiments and record results:

```sql
SELECT @@transaction_isolation;  -- REPEATABLE READ (default)

SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Deliverable: `concurrency_experiments.pdf` with screenshots of the failure and the fix (lab evidence for C5).

PHASE 9 (9): Indexing with EXPLAIN Evidence

```sql
-- Baseline: check the plan of the most common search

EXPLAIN SELECT  FROM products
```
```

```
WHERE category = 'FRUIT' AND unit_price <= 50000;

-- type: ALL  (full scan)  ->  record it


-- Fix with a composite index matching the filter pattern

CREATE INDEX idx_products_cat_price ON products(category, unit_price);

EXPLAIN SELECT  FROM products

WHERE category = 'FRUIT' AND unit_price <= 50000;

-- type: range  ->  record it


-- Buyer order history (used by "My Orders" page)

EXPLAIN SELECT  FROM orders WHERE buyer_id = 4 ORDER BY order_date DESC;

CREATE INDEX idx_orders_buyer_date ON orders(buyer_id, order_date DESC);


-- Product name search with FULLTEXT

ALTER TABLE products ADD FULLTEXT ft_product_name (product_name);

SELECT  FROM products WHERE MATCH(product_name) AGAINST('matooke');


-- Index cost awareness: measure insert speed before/after indexes (report it)
```

Deliverable: `optimization_report.pdf` before/after EXPLAIN screenshots (Assignment 4 material, minimum 2× documented improvement).

PHASE 10 (10–11): Optimization + Physical Design

Query rewrite: your team's monthly report is slow because of a function on the indexed column:

```sql
-- SLOW: function kills the index
SELECT  FROM orders WHERE YEAR(order_date) = 2025 AND MONTH(order_date) = 1;


-- FAST: sargable range
SELECT  FROM orders
WHERE order_date >= '2025-01-01' AND order_date < '2025-02-01';
```

Partitioning key design lesson: MySQL partitioned tables cannot have foreign keys, so we partition the denormalized analytics copy (this connects Phase 6's denormalization decision):

```sql
CREATE TABLE fact_order_lines (
  order_id    BIGINT NOT NULL,
  order_date  DATETIME NOT NULL,
  buyer_id INT, farmer_id INT,
  product_id INT, category VARCHAR(20),
  qty DECIMAL(10,2), unit_price DECIMAL(10,2), line_total DECIMAL(12,2),
  PRIMARY KEY (order_id, order_date)          -- partition key must be in PK
)
PARTITION BY RANGE (YEAR(order_date)) (
  PARTITION p2023 VALUES LESS THAN (2024),
  PARTITION p2024 VALUES LESS THAN (2025),
  PARTITION p2025 VALUES LESS THAN (2026),
```

```
  PARTITION pmax  VALUES LESS THAN MAXVALUE
);

-- Load history from the operational tables
INSERT INTO fact_order_lines
SELECT o.order_id, o.order_date, o.buyer_id, p.farmer_id,
       p.product_id, p.category, oi.qty, oi.unit_price, oi.line_total
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
              JOIN products  p ON p.product_id = oi.product_id;

-- Prove partition pruning: only one partition is scanned
EXPLAIN SELECT SUM(line_total) FROM fact_order_lines
WHERE order_date >= '2025-01-01' AND order_date < '2025-02-01';

-- Generate load to make benchmarks meaningful
DELIMITER //
CREATE PROCEDURE seed_history(IN n INT)
BEGIN
  DECLARE i INT DEFAULT 0;
  WHILE i < n DO
    INSERT INTO fact_order_lines (order_id, order_date, buyer_id, farmer_id,
                product_id, category, qty, unit_price, line_total)
    VALUES (900000 + i,
            NOW() - INTERVAL FLOOR(RAND()1000) DAY,
```

```
        1+FLOOR(RAND()50), 1+FLOOR(RAND()50),
        1+FLOOR(RAND()100), 'GRAIN',
        1+FLOOR(RAND()20), 5000, 5000);
    SET i = i + 1;
  END WHILE;
END //
DELIMITER ;
CALL seed_history(200000);
```

Deliverable: partitioned table + pruning evidence in the optimization report.

PHASE 11 (12): Replication Design (Distributed Module)

AgriConnect is growing; one server can't survive a crash or handle read traffic. Document (and, where possible, demo with two local MySQL instances/Docker):

```sql
-- On PRIMARY (my.cnf: server-id=1, log_bin=mysql-bin)
CREATE USER 'repl'@'%' IDENTIFIED BY 'StrongRepl1!';
GRANT REPLICATION SLAVE ON . TO 'repl'@'%';
SHOW MASTER STATUS;

-- On REPLICA (my.cnf: server-id=2)
CHANGE REPLICATION SOURCE TO
  SOURCE_HOST='127.0.0.1', SOURCE_PORT=3307,
  SOURCE_USER='repl', SOURCE_PASSWORD='StrongRepl1!',
```

SOURCE_LOG_FILE='mysql-bin.000001', SOURCE_LOG_POS=157;

START REPLICA;

SHOW REPLICA STATUS\G -- both threads must be Yes

Write one page answering: reads (reports) go to replica; writes (checkout) to primary; async replication risks stale reads, which reports tolerate, and which don't? Apply CAP reasoning.

PHASE 12 (13): NoSQL Extension the Flexible Catalog

Business rule 9: a cow needs `weight_kg`, milk needs `expiry`, grain needs `variety`. Instead of 30 nullable columns, use JSON:

```sql

ALTER TABLE products ADD COLUMN attributes JSON;

UPDATE products SET attributes = '{"variety":"Ssaali","organic":"yes"}' WHERE product_id = 1;

UPDATE products SET attributes = '{"breed":" Ankole","weight_kg":320}' WHERE product_id = 4;

UPDATE products SET attributes = '{"expiry_days":2,"cold_chain":"yes"}' WHERE product_id = 5;

-- Query inside JSON

SELECT product_name FROM products

WHERE JSON_UNQUOTE(JSON_EXTRACT(attributes,'$.organic')) = 'yes';

-- Index a JSON field via generated column
```

```
ALTER TABLE products
  ADD COLUMN organic VARCHAR(5)
    GENERATED ALWAYS AS (JSON_UNQUOTE(JSON_EXTRACT(attributes,'$.organic'))) VIRTUAL,
  ADD INDEX idx_organic (organic);

EXPLAIN SELECT product_name FROM products WHERE organic = 'yes';  -- now indexed
```

Add one page to the report: why the transactional core stays relational, while the catalog borrows a document pattern (technology selection justification, C9).

PHASE 13 (14): Final Assembly, Documentation & Defense

Final deliverable package (one Git repo):

#	***Artifact***	***Proves***
1	ERD (image + PDF)	C1
2	**schema.sql**, **seed.sql**	C1, C2
3	**normalization_audit.pdf**	C2
4	**queries_basic.sql** (15+), **queries_advanced.sql** (CTE/window/views)	C3
5	Relational algebra worksheet	C4
6	**procedures.sql** + rollback demo	C5
7	**concurrency_experiments.pdf**	C5
8	**optimization_report.pdf** (EXPLAIN, ≥2× speedup, partition pruning)	C6, C7
9	**replication_design.pdf**	C8
10	JSON catalog + tech-selection page	C9
11	15-minute demo + written report	C10

Demo script for presentation day (rehearse it):

1. Seed data → show catalog view.

2. `CALL sp_cash_in(...)` → wallet topped up.

3. `CALL sp_place_order(...)` → order created, stock reduced, both wallets updated, txn log written.

4. Force a failure (huge qty) → nothing changed (atomicity proof).

5. Two sessions buying the last item → concurrency proof.

6. Slow query → indexed query with EXPLAIN side-by-side.

7. Monthly dashboard query on partitioned table → instant.

Sample defense questions (be ready):

1. Why is `order_items.unit_price` frozen instead of joined from `products`?

2. Which ACID property does `SIGNAL` + `EXIT HANDLER` implement, and how?

3. Your indexes speed reads what do they cost? When would you remove one?

4. Why can't the operational `orders` table be partitioned directly?

5. If 10,000 buyers order simultaneously, what fails first and how would replication/sharding help?

6. Why did you keep the core relational and only the catalog semi-structured?

One-line summary for students

You are not "doing a database assignment"; you are building the transactional brain of a real agri-marketplace: model it (W2–3), build it (W4–6), make money movement safe (W7–8), make it fast (W9–11), scale it (W12–13), and defend it (W14). Every lecture has an immediate, visible payoff in your own system.